

Open URL Redirect Notes

Author: Eyad Islam El-Taher

May 31, 2026

1 Introduction

Open URL Redirect (also known as Open Redirect or Unvalidated Redirect) is a web application vulnerability that occurs when an application accepts user-controlled input and uses it as the destination of a redirect without proper validation.

This vulnerability allows attackers to craft URLs that appear to belong to a trusted domain while redirecting victims to attacker-controlled websites. Open Redirect vulnerabilities are commonly abused in phishing attacks, OAuth attacks, token theft scenarios, and as part of larger attack chains.

Note

Although Open Redirect is often classified as a low-severity vulnerability, its impact can become critical when chained with authentication mechanisms, OAuth flows, or social engineering attacks.

2 HTTP Redirection Status Codes

HTTP redirection responses belong to the 3xx class of status codes.

Code	Description
300	Multiple Choices
301	Moved Permanently
302	Found (Temporary Redirect)
303	See Other
304	Not Modified
305	Use Proxy
307	Temporary Redirect
308	Permanent Redirect

Note

Open Redirect vulnerabilities are most commonly observed in HTTP responses returning status codes 301, 302, 307, and 308 with a user-controlled `Location` header.

3 Redirect Methods

Applications may implement redirection using several methods.

3.1 HTTP Header Redirect

```
header("Location: https://example.com");
exit();
```

3.2 Path-Based Redirect

```
https://example.com/redirect/http://evil.com
https://example.com/redirect/../http://evil.com
```

3.3 JavaScript-Based Redirect

```
window.location = redirectUrl;
```

3.4 Meta Refresh Redirect

```
content="0;url=https://example.com">
```

4 Why Open Redirect Happens

Open Redirect vulnerabilities occur when developers trust user-controlled input and use it directly in redirection logic.

Common causes include:

- Lack of URL validation.
- Missing allowlists.
- Weak regular expressions.
- OAuth misconfigurations.
- Trusting GET or POST parameters.
- Client-side JavaScript redirects.

5 Vulnerable Code Examples

5.1 PHP Native

```
<?php
$url = $_GET['url'];

header("Location: " . $url);
exit();
?>
```

5.2 Laravel

```
public function redirect(Request $request)
{
    return redirect($request->url);
}
```

5.3 JavaScript

```
const redirectUrl =
new URLSearchParams(window.location.search)
.get("url");

window.location.href = redirectUrl;
```

Important

Never use user-controlled input directly in redirect functions.

6 Where to Find Open Redirects

Common locations include:

- Login pages
- Logout pages
- OAuth implementations

- SSO systems
- Password reset workflows
- Email verification pages
- Checkout systems
- Payment gateways
- Affiliate programs
- Tracking links
- Redirect endpoints
- Mobile deep links

7 Common Parameter Names

```
url=
redirect=
redirect_url=
redirect_uri=
redir=
next=
continue=
return=
return_to=
returnTo=
destination=
dest=
target=
go=
rurl=
checkout_url=
view=
```

8 Payloads

Common testing payloads:

```
https://evil.com
http://evil.com
//evil.com
///evil.com
https:evil.com
//evil.com
/%2f%2fevil.com
https://attacker.com
```

9 DOM-Based Open Redirect Example

Consider the following vulnerable JavaScript code:

```
<a href='#' onclick='returnURL =
/url=(https?:/.+).exec(location);

if(returnUrl)
location.href = returnUrl[1];
else
location.href = "/"'>
Back to Blog </a>
```

The application extracts the value of the `url` parameter from the current URL and directly assigns it to `location.href`.

Example:

```
https://victim.com/post?postId=4&
url=https://evil.com
```

When the victim clicks **Back to Blog**, the browser redirects to the attacker-controlled website.

Note

This vulnerability exists entirely on the client side and is therefore classified as a DOM-Based Open Redirect.

10 Methodology

1. Identify parameters that influence navigation.
2. Replace values with an external domain.
3. Observe redirection behavior.
4. Analyze HTTP response headers.
5. Test protocol-relative URLs.
6. Test encoded payloads.
7. Test JavaScript-based redirects.
8. Test OAuth redirect URIs.
9. Attempt filter bypass techniques.

11 Filter Bypass Techniques

11.1 Whitelisted Domain Bypass

```
[www.whitelisted.com.evil.com] (http://www.whitelisted.com.evil.com)
```

11.2 Protocol Relative URLs

```
//evil.com
///  
///evil.com
```

11.3 Bypass Blacklisted Slash

```
https:evil.com
```

11.4 Escaped Slash Bypass

```
//evil.com  
//evil.com
```

11.5 Null Byte Injection

```
[https://evil.com%00] (https://evil.com%00)
```

11.6 HTTP Parameter Pollution

```
?next=trusted.com&next=evil.com
```

11.7 Username@Host Trick

```
https://trusted.com@evil.com
```

11.8 Unicode Normalization

```
https://evil.c[U+2100].example.com
```

12 Impact

Possible impacts include:

- Phishing attacks
- Credential theft
- OAuth token theft
- Access control bypass
- Session theft
- Malware distribution
- Security filter bypass
- Brand reputation damage

13 Remediation

Recommended mitigations:

- Use allowlists for redirect destinations.
- Restrict redirects to internal paths.
- Validate hosts and schemes.
- Reject protocol-relative URLs.
- Validate OAuth redirect URIs exactly.
- Avoid user-controlled redirects whenever possible.
- Use indirect reference identifiers.